

HelixCode

HelixCode

작업을 분할하고, 보존하며, 작업 위치를 절대 잃지 않는 분산형 AI 개발 플랫폼

요약

HelixCode는 기업급 Go 기반 분산형 AI 개발 플랫폼으로, 작업을 SSH로 관리되는 워커 네트워크에 지능적으로 분배하며, 자동 체크포인팅과 롤백 기능을 통해 작업 내용을 절대 유실하지 않습니다. 다중 LLM 제공업체 통합, 전체 개발 라이프사이클 워크플로우, 크로스 플랫폼 배포를 REST, CLI, TUI, MCP 인터페이스로 통합 지원합니다.

간략 설명

HelixCode는 Go로 작성된 분산형 AI 개발 플랫폼입니다. SSH 기반 워커 네트워크에 작업을 지능적으로 분배하고, 자동 체크포인팅과 롤백으로 진행 상황을 보존하며, 다중 LLM 제공업체를 통합하고, REST, CLI, TUI, MCP 인터페이스를 통해 전체 개발 라이프사이클을 지원합니다.

상세 설명

HelixCode는 기업급 분산형 AI 개발 플랫폼(dev.helix.code, MIT 라이선스)으로, 슬로건이 명시하듯 단순한 약속을 실현합니다. 바로 '작업을 분할하고, 보존하며, 작업 위치를 절대 잃지 않는다'는 것입니다. 지능적인 작업 분할, 자동 작업 보존, 크로스 플랫폼 개발 워크플로우를 지원하며, 분산 컴퓨팅에 요구되는 동시성과 단일 바이너리 이식성을 위해 Go로 작성되었습니다. 자동 체크포인팅, 롤백, 실시간 모니터링은 선택적 부가 기능이 아닌 핵심 요소로 설계되었습니다.

이 플랫폼의 아키텍처는 REST + WebSocket + MCP로 구성된 API 레이어를 기반으로 하며, 핵심 서비스 세트로 구성됩니다. JWT 인증 및 세션 관리, 상태 모니터링이 포함된 SSH 기반 워커 풀 관리, 체크포인팅 및 의존성 처리가 가능한 작업 관리, 프로젝트 및 워크플로우 관리, 그리고 통합된 LLM 제공업체 레이어는 모두 PostgreSQL에 영구 저장되며, Redis는 선택적 조정 및 캐싱 계층으로 제공됩니다. 분산 워커는 네트워크를 통해 자동

설치되므로, 서버가 특정 머신을 가리키는 것만으로도 워커 확장이 가능하며, CLI, 터미널 UI, REST, 모바일 프레임워크 등 다양한 클라이언트 인터페이스를 통해 동일한 플랫폼에 접근할 수 있습니다.

HelixCode는 기획, 구축, 테스트, 리팩토링에 이르는 전체 개발 라이프사이클을 종합 지원합니다. 의존성 인식과 다중 세션 컨텍스트 추적을 통해 장기 실행 작업이 중단되거나 머신을 변경하더라도 작업 흐름이 유지됩니다. 또한 llama.cpp, Ollama, OpenAI 등 다중 LLM 제공업체를 단일 인터페이스로 통합하며, 사용 가능한 CPU/GPU/메모리를 감지해 모델을 최적화하고, 단일 패스로 해결하기 어려운 문제에 대해서는 사고 사슬(chain-of-thought) 및 사고 트리(tree-of-thoughts) 같은 고급 추론 전략을 지원합니다. Model Context Protocol은 표준화된 도구 및 컨텍스트 교환을 위해 다양한 전송 프로토콜로 구현되었으며, Slack, Discord, 이메일, Telegram 등 다중 채널 알림을 통해 분산 작업 진행 상황을 팀에 실시간으로 전달합니다. Linux, macOS, Windows, Aurora OS, SymphonyOS를 지원합니다.

콘텐츠

왜 만들었는가

분산 및 AI 지원 개발 환경에서는 작업이 여러 기기에 분산되거나 중단될 경우 맥락과 진행 상황이 유실되는 경우가 흔합니다. HelixCode는 작업을 능동적으로 분할하고 작업 상태를 자동으로 보존할 수 있도록 설계되었습니다. 이를 통해 대규모 개발 프로젝트를 세분화하고, 작업자 네트워크에 분배하며, 체크포인트를 설정하고, 상태를 잃지 않고 재개하거나 롤백할 수 있습니다.

혁신적인 이유

분산 AI 개발을 내구성 있게 만드는 것은 기존에 수작업으로 조각을 이어붙일 때는 결코 실용적이지 않았던 기능입니다. 세 가지 서로 다른 도구에서 각각 관리되던 기능이 하나의 플랫폼으로 통합되었습니다. 바로 분산 컴퓨팅(SSH 작업자 네트워크와 자동 설치 및 상태 모니터링), AI 개발 지원(다중 제공업체 LLM과 추론 및 도구 호출 기능), 그리고 전체 라이프사이클 워크플로 자동화입니다. 이 모든 것을 연결하는 핵심은 데이터베이스 기반 체크포인트링입니다. 작업 상태, 체크포인트, 의존성이 PostgreSQL에 영구 저장되기 때문에 여러 기기와 세션에 걸쳐 진행되는 작업도 중단된 지점부터 정확히 재개하거나 롤백할 수 있습니다. 중단과 분산 작업은 더 이상 진행 상황 손실의 원인이 되지 않고, 일상적이고 복구 가능한 사건으로 변합니다.

혁신적인 점

- 작업 보존을 핵심 요소로: 분산 개발 작업에 자동 체크포인트링과 롤백을 적용해, 중단이나 기기 장애가 발생해도 진행 상황이 사라지지 않고 유지됩니다.

- 하드웨어 인식 모델 선택: 감지된 CPU/GPU/메모리를 분석해 각 작업에 적합한 모델을 자동으로 매칭합니다. 작업자별 수동 조정이 필요 없습니다.
- 하나의 플랫폼, 다섯 개의 진입점: REST, WebSocket, CLI, TUI, MCP를 제공하며, MCP 자체도 다양한 전송 방식을 통해 노출되어 도구와 에이전트가 연결 방식에 관계없이 통합될 수 있습니다.
- 플랫폼 확장성: 일반적인 데스크톱 환경(Linux, macOS, 윈도우)을 넘어 Aurora OS와 SymphonyOS까지 지원해, 대부분의 도구가 무시하는 플랫폼까지 작업자 네트워크를 확장합니다.

가장 큰 기술적 도전 과제와 해결 방법

- 분산 및 중단 가능한 작업에서 진행 상황 유실 방지 작업이 여러 기기에 분산되면 충돌이나 중단이 발생할 경우 진행 중이던 작업이 고립되는 경우가 많습니다. 우리는 작업을 체크포인트와 의존성의 운반체로 모델링하고, 이를 PostgreSQL에 영구 저장함으로써 시스템이 마지막 정상 상태에서 롤백하거나 재개할 수 있도록 했습니다. 이는 메모리에 의존하는 취약한 상태가 아닌 데이터 계층에 내구성을 부여한 것입니다.
- 이종 작업자 네트워크 관리 Linux, macOS, 윈도우, Aurora, SymphonyOS 등 다양한 운영체제로 구성된 작업자 네트워크는 가용성과 설정이 끊임없이 변하는 대상입니다. 우리는 전용 작업자 풀 서비스를 통해 SSH 기반 등록, 새 노드 자동 설치, 지속적인 상태 모니터링을 수행해 네트워크가 기기의 출입에 관계없이 항상 파악되고 제어될 수 있도록 했습니다.
- 제공업체 및 하드웨어 이질성 문제 LLM 백엔드와 이를 실행하는 기기는 성능과 사양이 천차만별입니다. 우리는 통합된 LLM 제공업체 인터페이스를 통해 이러한 차이를 숨기고, 하드웨어 감지(CPU/GPU/메모리) 기능을 결합해 지능적인 모델 선택을 구현했습니다. 이를 통해 호출자는 제공업체나 하드웨어를 고려할 필요 없이 적합한 모델이 적합한 기기에 자동으로 할당됩니다.

콘텐츠

기술 스택

- **Go (1.26+ 내부 모듈)** — 고루틴 기반 동시성과 단일 바이너리 출력이 분산 워커 시스템에 필요한 요소이기 때문에 선택되었습니다. 병렬 오케스트레이션을 저렴하게 제공하며, 자체 포함 바이너리는 모든 노드에 자동 설치됩니다. 모든 핵심 서비스와 CLI/서버 바이너리를 포함합니다.
- **Gin (HTTP 프레임워크)** — 빠른 속도와 낮은 오버헤드를 갖춘 최소한의 REST 레이어를 제공하기 위해 선택되었습니다. 모든 클라이언트가 사용하는 /api/v1 인터페이스(인증, 워커, 작업, 프로젝트)를 제공합니다.
- **PostgreSQL 15+ (pgx/v5 사용)** — 내구성 있는 시스템 기록을 위해 선택되었습니다. 체크포인트링과 롤백을 위한 트랜잭션 지속성이 요구되기 때문입니다. 11개 테이블로 구성된 분산 컴퓨팅 스키마(사용자, 워

커, 작업, 프로젝트, 세션, llm_providers, 알림)를 관리하여 작업 보존을 가능하게 합니다.

- **Redis 7+ (선택 사항, go-redis/v9)** — 선택적 캐싱 및 조정 계층으로 선택되었습니다. 핫 패스를 가속화하면서도 필수 의존성이 되지 않도록 하여, 최소한의 배포 환경에서는 PostgreSQL만으로도 실행 가능합니다.
- **SSH** — 이미 널리 사용되며 보안성이 검증된 프로토콜이기 때문에 워커 제어 전송 계층으로 선택되었습니다. 별도의 에이전트 배포 없이 전체 풀에 걸쳐 워커 등록, 자동 설치, 원격 명령 실행을 지원합니다.
- **Model Context Protocol (MCP)** — 표준화된 도구 및 컨텍스트 교환을 위해 선택되었습니다. 외부 도구와 에이전트가 하나의 개방형 프로토콜을 통해 통합되며, 클라이언트가 연결된 환경에 맞춰 다중 전송을 지원합니다.
- **LLM 제공자(llama.cpp, Ollama, OpenAI)** — 로컬 및 호스팅 추론을 하나의 통합 인터페이스로 제공하기 위해 선택되었습니다. 호출자는 모델의 위치를 알 필요 없이 하드웨어 인식 라우팅을 통해 작업이 로컬 모델 또는 호스팅 모델로 전달됩니다.

현황 및 솔직 공지

- **현황:** 베타. README에는 “완전히 완성 / 5단계 모두 완료” 상태라고 명시되어 있으나, 이는 프로젝트 자체 주장일 뿐 독립적인 검증은 이루어지지 않았으므로 이 페이지에서는 베타로 간주합니다.
- 위의 모든 세부 사항은 저장소 README를 참고한 것이며, 마케팅 문구(태그라인)는 편집자의 의견일 뿐 실제 지표는 아닙니다.

우선 순위 계층: Helix-주요.